顺序表与双指针暴力解模板

408 | 数据结构 · 左右指针逆序、辅助数组归并暴力解、暴力循环与最优解对比

一、双指针基础母板：左右指针原地逆序

408 考场最基础的左右夹逼双指针，用于数组原地反转、前后段对称对调。

```
/* 零件：数组 A[0..n-1] 原地逆序（左右指针夹逼互换） */
void ReverseArray(int a[], int n) {
    int i, j, temp;
    for (i = 0, j = n - 1; i < j; i++, j--) {
        temp = a[i]; a[i] = a[j]; a[j] = temp;   /* 发生互换 */
    }
}
/* 变体：逆序指定区间 A[left..right] */
void Reverse(int a[], int left, int right) {
    int i = left, j = right, temp;
    while (i < j) {
        temp = a[i]; a[i] = a[j]; a[j] = temp;
        i++; j--;
    }
}
```

二、通用暴力通杀模板：辅助数组归并（两序列合并）

考场通杀场景：2011 寻找两升序序列中位数、两有序表合并、集合求交并集。

暴力解思想：无需推导数学淘汰规则，直接 malloc 一个大小为 $n_1 + n_2$ 的辅助数组，双指针遍历比大小归并，做完直接按下标取答案。考场稳拿 11~12 分（仅扣少量空间分）！

```
/* 通用母板：两有序数组合并到辅助数组 temp（暴力通杀 2011 中位数等大题） */
void MergeArrays(int a[], int n1, int b[], int n2, int temp[]) {
    int i = 0, j = 0, k = 0;
    while (i < n1 && j < n2) {          /* 主循环：两边都没遍历完 */
        if (a[i] < b[j]) temp[k++] = a[i++];
        else temp[k++] = b[j++];
    }
    while (i < n1) temp[k++] = a[i++];  /* 收尾：a 还有剩余 */
    while (j < n2) temp[k++] = b[j++];  /* 收尾：b 还有剩余 */
}

/* 2011 真题暴力解：求两等长升序序列 A、B 的等效中位数 */
int FindMedian_Brute(int a[], int b[], int n) {
    int *temp = (int *)malloc(sizeof(int) * (2 * n));
    MergeArrays(a, n, b, n, temp);      /* 一键归并到辅助数组 */
    int ans = temp[n - 1];              /* 第 n 个元素（下标从 0 起算为 n-1）即中位数 */
    free(temp);
    return ans;                         /* 时间 O(n)，空间 O(n)，考场稳拿 11 分 */
}
```

最优解对比（**2011**）：归并双指针只向前推进计数 n 次，用单个变量 median 记录较小值而不实际开辟 $2n$ 数组，时间 $O(n)$，空间优化为 $O(1)$。

三、通用暴力通杀模板：开辅助数组暂存实现循环位移（**2010** 真题）

题型特征：将长度为 n 的数组循环左移（或右移）p 个位置。

暴力解思想：开辟大小为 p 的辅助数组暂存移出部分，剩余元素整体前移，再将暂存数组拼回末尾。极其直观，绝不写错！

```
/* 2010 真题暴力解：开辅助数组暂存实现循环左移 p 位 */
void LeftShift_Brute(int a[], int n, int p) {
    p = p % n;                          /* 关键防越界：p 可能大于等于 n */
    if (p <= 0) return;
    int *temp = (int *)malloc(sizeof(int) * p);
    int i;
    for (i = 0; i < p; i++) temp[i] = a[i];             /* 1. 暂存前 p 个元素 */
    for (i = p; i < n; i++) a[i - p] = a[i];             /* 2. 后面 n-p 个整体前移 */
    for (i = 0; i < p; i++) a[n - p + i] = temp[i];     /* 3. 暂存拼回末尾 */
    free(temp);                                         /* 时间 O(n)，空间 O(p) */
}
```

最优解对比（**2010** 三次逆置）：`Reverse(a, 0, p-1); Reverse(a, p, n-1); Reverse(a, 0, n-1);`，先分后合三次逆置，时间 $O(n)$，空间 $O(1)$。

四、辅助标记数组暴力解：未出现最小正整数（**2018** 真题）

暴力解思想：数组开大一点（开 $n+2$ 大小充当哈希桶），扫描原数组打标记，再从正整数 1 往后查第一个没被标记的位置。

```
/* 2018 真题暴力解：辅助桶标记找未出现最小正整数 */
int FindFirstMissing_Brute(int a[], int n) {
    int *hash = (int *)malloc(sizeof(int) * (n + 2));
    int i;
    for (i = 0; i < n + 2; i++) hash[i] = 0;            /* 必须手动清零 */
    for (i = 0; i < n; i++) {
        if (a[i] >= 1 && a[i] <= n + 1) hash[a[i]] = 1; /* 命中范围打标记 */
    }
    for (i = 1; i <= n + 1; i++) {
        if (hash[i] == 0) { free(hash); return i; }     /* 找到答案直接返回 */
    }
    free(hash);
    return n + 2;                                       /* 时间 O(n)，空间 O(n) */
}
```

五、多重循环暴力解 **vs** 贪心双指针（**2013/2020** 真题）

1. 2013 主元素查找

- 暴力解（双重 `for` 循环统计）：外层枚举每个元素，内层统计它在数组中的出现频次，若超过 $n/2$ 则返回。时间 $O(n^2)$，空间 $O(1)$。保底拿 10 分！
- 最优解（摩尔投票法）：一次遍历利用 `count` 与 `candidate` 对消，第二趟验证。时间 $O(n)$，空间 $O(1)$。

2. 2020 三元组最小距离 $D=|a-b|+|b-c|+|c-a|$

- 暴力解（三重 `for` 循环暴力穷举）：三层嵌套直接算出所有组合的 D 取最小值。代码仅 8 行，时间 $O(n_1n_2n_3)$，空间 $O(1)$。
- 最优解（三指针贪心步进）：利用 $D=2(\max-\min)$，每次仅步进指向最小值的指针。时间 $O(n_1+n_2+n_3)$，空间 $O(1)$。

```
/* 2020 三元组暴力解（考场兜底秒写，时间 O(n^3)，空间 O(1)） */
int MinDist_Brute(int a[], int n1, int b[], int n2, int c[], int n3) {
    int minD = 0x7fffffff, curD, i, j, k;
    for (i = 0; i < n1; i++) {
        for (j = 0; j < n2; j++) {
            for (k = 0; k < n3; k++) {
                curD = abs(a[i]-b[j]) + abs(b[j]-c[k]) + abs(c[k]-a[i]);
                if (curD < minD) minD = curD;
            }
        }
    }
    return minD;
}
```

六、考场答题纪律与得分口径

1. 暴力解能拿多少分：408 算法题满分 13~15 分。若能把暴力解的“设计思想、核心代码、时空复杂度”三段完整写出，评卷标准统一保底给 **10~11** 分。
2. 动笔策略：考场 5 分钟内想不出 $O(n)$ 最优解，立刻转写辅助数组/归并/多重循环暴力解。千万不要花半小时苦思冥想最优解最后代码写不完得 0 分。
3. 开数组铁律：严禁 `int temp[n]`（变长数组编译报错），一律写 `(int *)malloc(...)`，开完立刻循环清零，用完必须 `free`。